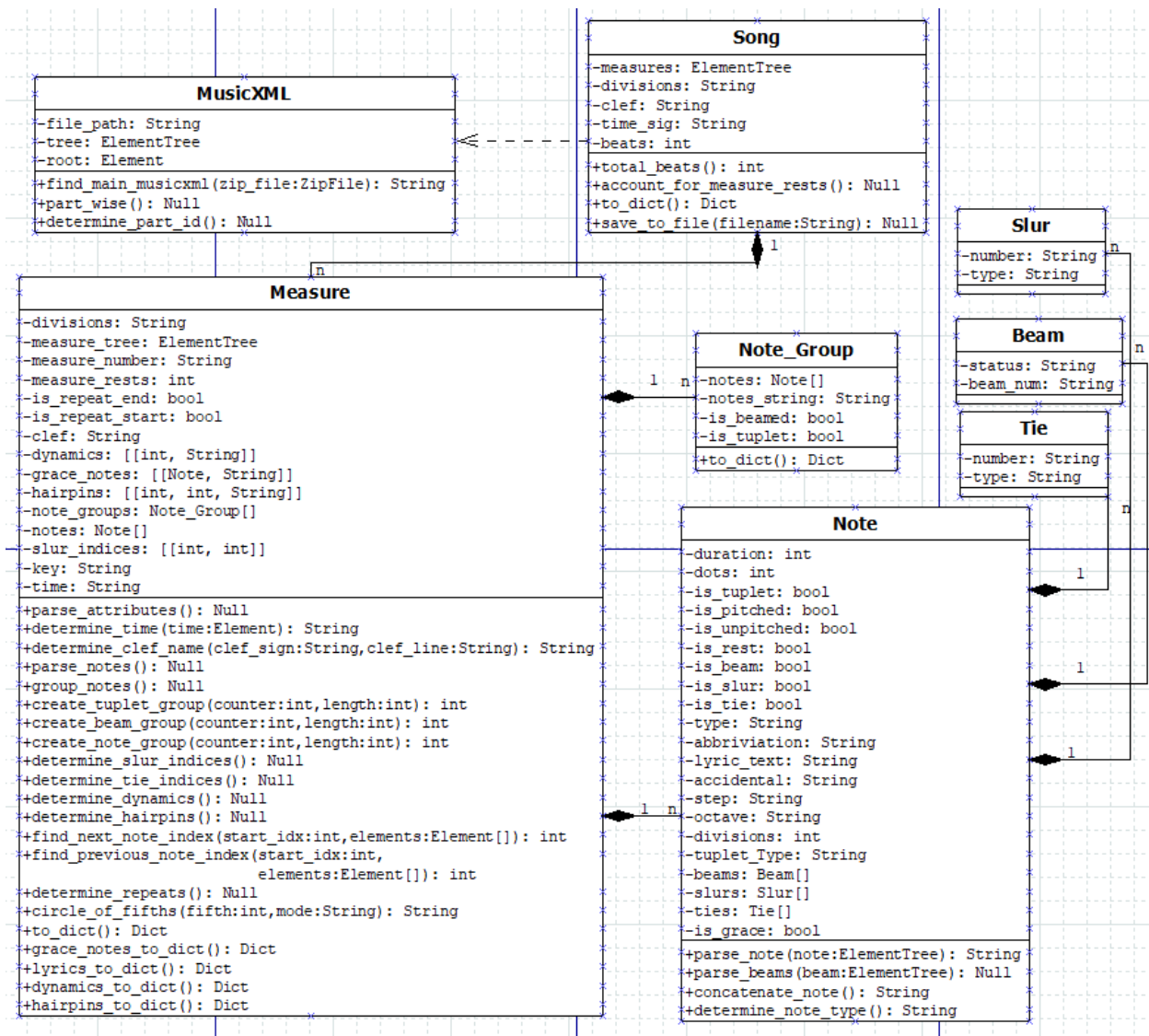




Introduction

The parser is designed using an object-oriented architecture using container classes. At the peak of this architecture is the MusicXML class, which acts as the entry point for processing mxl (compressed MusicXML) files. This class handles the extraction of XML data by unzipping the compressed file structure and loading that into an XML Element Tree. From there, important musical elements from the Element tree such as the measure element are extracted and passed into the Song class. This class stores

data such as time signatures, key signatures, clef and beat information, and manages an array of Measure objects.

Each Measure instance encapsulates data specific to a musical measure such as, divisions, repeats, dynamics, hairpins, note groups and more. It also parses individual notes within the measure, creating and storing Note objects.

The Note class stores a single musical note and contains fields for pitch, duration, type, and more. Additionally, the Note class stores arrays of Beam, Slur and Tie objects to support multiple instances of these features, such as nested slurs. The Note_Group class is also used by Measure to group notes accordingly based on if they are beamed or a part of a tuplet.

Finally, all of the data within these container classes are formatted to dictionaries based on the values of their unique attributes to be compiled into a JSON file which goes to the render.

Directory Structure

At the root of the directory holds the main parser.py and an example JSON output file. There then four subdirectories. The UnitTest folder holds all individual unit test files along with a test_suite file to run all tests at once. The UnitTestFiles folder holds “off the shelf” mxl files used for testing along with some example musicxml files. The CustomUnitTestFile folder contains all custom mxl files created by me. Each file has a .mxl file type for testing and .mscz used to view the sheet music in MuseScore Studio. Finally there is a Conversion XSL File folder that contains a parttime.xsl file useful for converting a time-wise musicxml file into a part-wise file.

Imported Libraries

ZipFile

The zipfile library is a useful python library used for reading and writing to zipfiles. In this script the library is used to read from the compressed musicxml file (.mxl) as the XML file that contains all of the song data is within the zip structure.

xml.etree.ElementTree

The element tree library is used for converting a XML document into a n-ary tree structure (ElementTree) where each Element object is represented as a node in the data structure. In this script the library is used to convert the main XML file to a tree data structure to more easily and clearly store the data of the song into the container classes. Below are some commonly used functions in the script.

parse() will parse a XML document into a ElementTree

```
self.tree = ET.parse(xml_file)
```

getroot() will return the root element of the document, in the case of a musicxml file this is one of two values, part-wise or time-wise. These elements indicate how the file is organized and structured. A part-wise piece will contain all measures of the song within each part, specific to each part. While a time-wise file will contain each part within a measure for all measures of the song.

```
self.root = self.tree.getroot()
```

The find(String) function will search through the tree and return the element specified or None if the element does not exist. If the string of the function is preceded by './..' then the function will search through all descending layers of the tree while without it the function will only search immediate children elements. More backslashes can be added to search within specific children elements like a directory

```
rootfile = tree.find('./rootfile')
```

The `findall(String)` function will return all elements matching the given string in an array or an empty array if no elements are found. Same functionality with the `'./'` preceding string and extra backslashes.

```
slurs = note.find('notations').findall('slur')
```

The `.text` and `.attrib` variables will return the text of the element or all attributes of the element in a dictionary.

```
self.step = note.find('pitch/step').text  
self.tuplet_type = note.find('notations').find('tuplet').attrib.get('type')
```

The `.tag` variable will return a string of the element name.

```
if element.tag.lower().endswith("direction"):
```

Json

The `json` library is used in this script to write the container class dictionaries to a `json` file.

Decimal

The `Decimal` library is used in this script when calculating the note type based on the durations to ensure if there are dots involved to not round up to the next note if there are enough dots to manage that.

Sys

The `sys` library is used for error handling and catching within the script.

Typing

The `List` and `Tuple` libraries from the `typing` library are used in this script to clarify what data types a variable or array should hold. While this library does not enforce the variable to comply with this structure it is useful to show what functions are being called from which objects, especially with the custom container classes.

Cade LaVanchy

Constant Data Structures

Key Signature Arrays

Four ordered lists represent the circle of fifths for both major and minor keys. These arrays assist in identifying key contexts and in rendering harmonic relationships:

- `major_sharp`, `major_flat`: Major keys with sharps and flats.
- `minor_sharp`, `minor_flat`: Minor keys with sharps and flats.

Rest Placement Dictionary

The `rest_placement` dictionary provides default vertical positions for rest symbols on the staff when their display location is not explicitly defined in the XML. Each entry maps a note type (e.g., "whole", "quarter") to a corresponding staff position (e.g., "d5", "b4").

Note Abbreviation Dictionary

The `note_abbreviations` dictionary converts MusicXML note type strings into shorthand notations suitable for rendering. For example, "quarter" becomes "q" and "16th" becomes "16".

Accidental Mapping Dictionary

The accidental dictionary translates MusicXML accidental values into symbols used by the rendering engine. This includes standard accidentals like sharp (#) and flat (b), as well as extended notations like double-sharp (##) and natural-flat (nb).

Clef Mapping Dictionaries

- `clef_map`: Maps a clef symbol (e.g., "G", "C") and staff line number to a named clef (e.g., "treble", "tenor"), ensuring accurate clef recognition based on both properties.

- `default_clefs`: Provides default clef names when only a clef symbol is given, enabling fallback logic for incomplete clef definitions in the input XML.

Container Classes

MusicXML

Description

The MusicXML class parses through the mxl compressed zip format to locate the main zipfile. MusicXML has documentation that describes that every compressed zip file should contain a `container.xml` file within a META-INF folder that states the file path of the main xml file. This class locates the main xml file by parsing through the `container.xml` file, if the file is not found it will default to looking through the root directory for a xml or musicxml file. Once this is done the `ElementTree` and root are determined along with the part id. Every `<part>` element has a unique id and that is determined by finding the part with lyrics attached to the notes. Once the part id is found an array of `<measure>` Elements are passed into the Song class.

Features

- Locate main xml file within zip structure
- Determine vocal part based on lyrics
- Determine list of measure Elements for part-wise xml file structure

Current State

The current state of the class supports unzipping a mxl file that is formatted part-wise and storing that file in the tree data structure which passes the measure elements into the Song class. Currently there is no support for time-wise formatted xml files but in testing and according to MusicXML's documentation the format option is likely going to be depreciated and most prominent digital sheet music applications that support mxl files export in part-wise format anyways. To add support for timewise files

within the parsers directory there is a Conversion XSL Files folder that contains a parttime.xsl file that along with the lxml python library can convert a part-wise file to a time-wise file.

Testing

Currently the testing that has been done is to check for unsupported file extensions and variations of the supported file extensions.

Song

Description

This class is responsible for managing all Measure objects and performing several key tasks to ensure accurate musical representation. If a <type> element is missing from a note, it calculates the note type based on duration. The divisions value from the first measure is propagated forward to subsequent measures, ensuring each measure has a valid divisions value. If a later measure defines a new divisions value, the class updates accordingly.

Additionally, it calculates the total number of beats in the song using the time signatures found throughout the piece. This total is used by the renderer and grader to determine the tempo of the music.

The class also handles multi-measure rests by adjusting the list of measures. When a measure includes a multi-measure rest, the subsequent measures—equal to the rest's duration—often contain redundant whole rests. To correct this overcount, the class removes those extra measures.

Finally, it formats all relevant information and internal variables into a dictionary, which can then be exported as a JSON file. The file needs to be encoded to “utf=8” and ensure_ascii set to false to ensure special characters in lyrics are written to the JSON as the actual characters and not their ASCII equivalents.

Features

- Converts MusicXML <measure> elements into structured Measure objects.
- Cascades divisions values from previous measures unless explicitly updated.
- Extracts and stores the initial clef and time signature.
- Supports time signature changes across measures.
- Computes total beats across the score based on each measure's time signature.
- Handles multi-measure rests by removing redundant placeholder measures.
- Stores key signatures at the measure level to support key changes.
- Exports all relevant data (clef, time sig, key sig, measures, total beats) to a JSON file.

Current State

The Song class is designed to serve as a high-level container for musical structure and metadata. It takes as input a list of MusicXML <measure> elements—retrieved using the MusicXML class—and converts them into a sequence of structured Measure objects. The class preserves essential musical attributes such as clefs, time signatures, and divisions, cascading values like divisions from earlier measures to later ones unless explicitly redefined. The clef and initial time signature are stored for global reference, while each measure retains its own local time signature, allowing the class to compute the total number of beats across the song accurately, even when time signature changes occur mid-score. Key signatures, which are required for both rendering and grading, are handled on a per-measure basis to account for key changes. The class also manages multi-measure rests by removing redundant placeholder measures that would otherwise lead to overcounting. After processing, all relevant data is formatted into a dictionary structure and can be exported to a JSON file for downstream use.

This class operates in tandem with the MusicXML class, which is responsible for unzipping .mxl archives, parsing the internal MusicXML content, and identifying the appropriate part to process.

Specifically, the MusicXML class supports multipart scores by scanning for the part that contains vocal elements—identified by the presence of `<lyric>` tags—ensuring that the vocal line is selected for parsing. While the Song class currently focuses on the vocal part, the architecture supports multipart files and can be extended to include additional parts if needed.

Testing

Currently most of the testing is for calculations and data storage in the Measure and Notes class which relies on the Song class. There is testing for the total beats and multi-measure rests, most of the testing is implicitly done through the results and testing of the other container classes.

Measure

Description

This class serves as a container and parser for individual measures within a MusicXML file, extracting structural and performance-related data into a normalized format suitable for downstream rendering or analysis. Upon initialization, it processes the XML element tree of a measure to populate internal representations of notes, grace notes, slurs, ties, dynamics, hairpins, and repeat indicators. Slurs and ties are tracked by matching start and stop tags via number attributes, allowing multi-note linkages to be resolved across the note sequence. Dynamics are mapped to their corresponding note indices by locating `<direction>` elements and associating them with the nearest note following their appearance in the XML tree. Similarly, crescendo and diminuendo wedges (hairpins) are parsed with respect to their start and stop types and resolved to note-relative indices. Repeat signs and key signatures are interpreted using MusicXML's structural tags and contextualized via the circle of fifths to output key names. Note are group by indicators like if they are beamed or in a tuplet The class provides serialization methods (`to_dict` and auxiliary formatting helpers) to transform internal state into JSON-compliant dictionaries, making it

well-suited for integration with the render. Through its modular parsing and mapping strategy, the class abstracts complex MusicXML semantics into a clean, object-oriented interface.

Features

Notes & Grace Notes

- **Note Parsing:** Extracts standard `<note>` elements, converts them into internal note objects with duration, pitch, accidentals, and octave information.
- **Grace Notes Support:** Detects `<grace>` tags and stores associated grace notes with the index of the main note they attach to.
- **Tuplet Recognition:** Differentiates between normal notes and triplets, including the number of notes within each triplet group.
- **Beam Recognition:** Differentiates between normal notes, beams, and triplets to group the notes accordingly. If there are beamed notes that are semi overlapped by a triplet group the beamed group will be split into two beamed groups to prioritize the triplet group.
- **Lyric Association:** Associates lyrics (from `<lyric>`) with the correct note or triplet, and supports formatting for render compliance.
- **Note Serialization:** Converts each note group into a structured format via `to_dict()` including keys, durations, and special flags (like `is_triplet` or `is_grace`).

Slurs & Ties

- **Slur Detection:** Parses `<slur>` elements, resolving start and stop tags using the number attribute to link note indices correctly.
- **Tie Detection:** Identifies `<tie>` elements (including multi-measure ties), adds them to `slur_indices` for compatibility with render tools.

- **Unmatched Slur/Tie Handling:** Supports incomplete slurs or ties by marking unresolved endpoints as None.

Dynamics

- **Explicit Dynamics Parsing:** Captures dynamics from <direction><dynamics>, supporting compound dynamics like fp, sfz, etc.
- **Dynamic-to-Note Mapping:** Associates each dynamic with the next occurring note in the measure tree.
- **Tuplet-Aware Formatting:** Calculates tuplet grouping for dynamic-linked notes for accurate rendering.
- **Dynamic Serialization:** Exports parsed dynamics as a list of objects with fields for text, duration, and tuplet context.

Hairpins (Wedges)

- **Hairpin Detection:** Parses <direction-type><wedge> elements for crescendo and diminuendo types.
- **Index Resolution:** Maps start and stop wedge points to the appropriate note-relative indices.
- **Fallback Strategy:** Uses both forward and backward search to find associated notes when wedges are not directly aligned.
- **Hairpin Serialization:** Exports a list of objects with start and end note indices and hairpin type (1: crescendo, 2: diminuendo).

Repeats & Measure Metadata

- **Repeat Sign Detection:** Checks for <repeat> elements with direction="forward" or direction="backward" and sets is_repeat_start or is_repeat_end.

- **Measure Number Tracking:** Stores the XML-assigned measure number for indexing and sequence tracking.
- **Measure Rests:** Detects whole-measure rests to flag empty measures structurally.

Key Signature & Time Signature

- **Key Signature Resolution:** Converts <fifths> and <mode> into human-readable keys via the circle of fifths logic.
- **Time Signature Parsing:** Extracts <time> elements to determine the time signature of the measure (e.g., 4/4, 6/8).

Utility & Export

- **Full JSON Export:** Aggregates all structured data into a unified dictionary (to_dict) for easy serialization and transmission.

Current State

The current implementation of the Measure class offers a comprehensive and modular approach to parsing and structuring MusicXML data at the measure level. It successfully extracts a wide variety of musical elements, including notes, rests, grace notes, tuplets, slurs, ties, dynamics, hairpins, lyrics, and repeat markings. These features are systematically stored in internal data structures and exported via a to_dict() method for downstream rendering. Slurs and ties are resolved using slur or tie numbers and stored as index pairs; dynamics are mapped to notes based on position in the XML tree; hairpins are interpreted with fallback logic to ensure both start and stop points are captured; and key signatures are resolved using circle-of-fifths mappings for both major and minor modes. The class is designed to handle various nuances of note grouping and ensures render-compliant output by incorporating contextual logic for tuplets, lyrics, and dynamics.

However, several limitations remain in the current design. The class is primarily tailored to single-voice measures, and may not correctly parse or disambiguate elements in multi-voice contexts. Slurs and ties that span across measure boundaries are only partially supported, as slurs across bars are not supported, the code for that support is found commented out due to no render support for slurs across measures. This restricts full phrase reconstruction without higher-level context. While grace notes and tuplets are supported, more complex tuplet scenarios—such as nested tuplets or multiple grace notes attached to a single note—are not fully implemented. Additionally, expressive notations like articulations, ornaments (e.g., trills, mordents), and advanced playback instructions are not currently captured. These limitations suggest that while the class provides a strong foundation for MusicXML parsing, it requires further expansion to support the full richness and complexity of modern music notation.

Testing

This class encapsulates the majority of the testing of the parser script. All features described above are tested for functionality and limitations within individual test files. These limitations include testing for unsupported feature details like unsupported clef or key signatures. In the parser script when there is support for a feature but the feature does not fully cover all possible values, defaults are made as a backup. Examples include defaulting to treble clef if the clef mode is unrecognized or defaulting to a fractional time signature if the time signature symbol is not supported.

MusicXML

Description

The Note class serves as the foundational unit for representing and manipulating individual `<note>` elements from MusicXML within the parser. Upon initialization, it parses an XML note element and

encapsulates all relevant musical properties into structured attributes for use in downstream rendering and logic. This includes the pitch, duration, rhythmic type, beams, slurs, ties, accidentals, dynamics, tuplets, lyrics, and grace note status.

The class supports a flexible representation of both pitched and unpitched notes, as well as rests, and makes clear distinctions between each through internal boolean flags. When parsing a note, the class first establishes its basic type and duration, followed by specific musical modifiers like dot count, accidentals, or tuplets. Composite notations such as beams, slurs, and ties are mapped into structured sub-objects (Beam, Slur, Tie) and stored in lists to accommodate multiple overlapping relationships. The class also handles lyrics with support for syllabic and elision formatting and serializes all note data into a standardized string format (`note_string`) for compatibility with the rendering engine.

The class includes specialized logic to account for grace notes, implicit rests (using fallback placements), and tuplets derived from both `<tuplet>` and `<time-modification>` tags, while excluding unsupported elements like tremolos. Additionally, utility methods are provided to access internal properties, allowing external components to query a note's musical function and structural role within a measure.

Features

Pitch and Register

- Determines whether the note is pitched, unpitched, or a rest based on `<pitch>`, `<unpitched>`, and `<rest>` tags.
- Extracts the musical step (e.g., C, D, E) and octave number for pitched and unpitched notes.
- For rests, provides fallback step and octave using a predefined `rest_placement` dictionary if display information is not explicitly provided.

Duration and Type

- Parses the `<duration>` element to determine the length of the note.
- Computes the rhythmic type of the note (e.g., quarter, half, sixteenth) from the duration and divisions if the `<type>` element is not provided.
- Supports the use of the `<dot>` element to count rhythmic augmentation dots.
- Recognizes and assigns a duration to grace notes using fallback logic when no duration is explicitly specified.

Accidentals

- Extracts and stores accidental information from the `<accidental>` tag (e.g., sharp, flat, natural).
- Converts MusicXML accidentals into renderable notation symbols for output formatting.

Beams

- Identifies beamed notes via `<beam>` elements.
- Stores beam information as a list of Beam objects, each containing beam number and status (e.g., begin, continue, end).
- Provides access to beam-level information for rendering and playback purposes.

Tuplets

- Detects triplets through `<notations>/<triplet>` elements and, alternatively, via the `<time-modification>` element when no `<tremolo>` is present.
- Captures the triplet type attribute (start, stop, single) and stores it for logical grouping of notes within triplets.

Slurs

- Extracts slur markings from `<notations>/<slur>` elements.

- Stores each slur as a Slur object with number and type (start or stop).
- Supports multiple slurs on the same note and maintains their respective identities.

Ties

- Detects ties from <notations>/<tied> elements.
- Supports the possibility of multiple simultaneous ties and stores them as Tie objects containing the tie number and type (start or stop).
- Enables tracking of cross-measure ties and tie continuity.

Lyrics

- Extracts lyric text from <lyric>/<text> elements.
- Handles syllabic divisions such as "begin", "middle", and "end", adding appropriate punctuation (e.g., hyphens) from the <syllabic> element.
- Accounts for elisions via <elision> tags and appends double hyphens where appropriate.
- Concatenates all lyric components into a unified lyric_text string for rendering or analysis.

Note String Construction

- Generates a standardized note string for rendering (e.g., c#4/q.).
- Adjusts formatting for rests (e.g., c4/q/r) and appends dot notation as needed.
- Converts the note type to its abbreviated form based on a mapping dictionary, with fallback to quarter note if the type is unsupported.

Internal Flags and Metadata

- Maintains boolean flags indicating:
 - Whether the note is a rest, pitched, unpitched, grace note
 - Whether the note is beamed, part of a tuplet, slurred, or tied

- Stores the note abbreviation for use in output and rendering contexts.

Accessor and Utility Methods

- Includes logic to derive note type from duration and divisions using decimal arithmetic to ensure precision.

Current State

The current state of the Note class reflects a comprehensive and modular approach to parsing and representing individual musical notes from MusicXML data. It supports a wide array of features, including pitch and register identification for pitched, unpitched, and rest notes, along with fallback logic for missing display attributes in rest elements. Duration parsing is well-supported, including rhythm type inference when the `<type>` tag is absent, and dot counting is handled through direct counting of `<dot>` elements. The class distinguishes grace notes and assigns a fallback duration, though this may be inaccurate if the source score varies in grace note treatment. Accidental information is parsed and integrated into a consistent renderable format. Structural annotations such as beams, tuplets, slurs, and ties are recognized and stored in respective object lists, which provides flexibility for rendering and analysis. However, the tuplet logic currently conflates `<time-modification>` with tuplets while excluding tremolos, which may lead to false positives or overlook edge cases in complex scores. Slurs and ties are robustly supported, even in overlapping cases, but interpretation of their scope across measures may be limited without a broader context. Lyric handling accounts for syllabic and elision markers, producing clean concatenated strings, though it assumes basic syllabic logic and lacks support for extended lyrics. The Note class also includes a well-defined note string formatter, which adapts for rest notation and dotting, but could be further refined to reflect nested articulations or ornaments. Overall, while the class provides strong foundational support for most common MusicXML features, it does not yet handle nuances such as articulations, ornaments, or multi-voice conflicts, and its behavior across measure boundaries is bounded by the scope of the individual note element.

Testing

Like the Measure class the other major portion of testing was done for attributes of the note class, such as testing note types like pitched vs unpitched, note types, accidentals, slurs, ties and dotted note functionality. It should be important to note that while there is support for 128th notes, the software used to create custom xml files MuseScore Studio does not support 128th notes so that functionality is untested. Also there is a limited amount of accidentals supported and will default to no accidental if the symbol in the <accidental> element is unrecognized.

Helper Container Classes

Slur, Tie, Beam, Note_Group

Description

The minor container classes in this parser—Note_Group, Beam, Slur, and Tie—serve as lightweight, modular data structures for organizing and annotating musical note relationships extracted from MusicXML. Each class plays a targeted role in supporting specific notation contexts. The Note_Group class is designed to encapsulate collections of Note objects that share common structural flags, such as being part of a beam or a tuplet. It tracks both the group composition and relevant attributes (is_beamed, is_tuplet) and provides formatted output through a dictionary representation for downstream processing or JSON serialization.

The Beam, Slur, and Tie classes each encapsulate directional and numerical metadata for their respective notational elements. The Beam class records the beam number and status (start, continue, or stop), enabling multi-level beaming analysis across rhythmic groupings. The Slur and Tie classes similarly capture the type and identification number of each slur or tie, which is crucial for interpreting multi-note phrasing, articulation, or duration extension. These classes are designed with minimal overhead and clean getter methods, making them ideal for use in lists or as annotations attached to notes.

Together, these classes support a compositional approach to note-level semantics, enabling higher-level analysis and rendering logic while maintaining a clear separation of concerns. Their simplicity and focused scope allow them to scale cleanly and integrate seamlessly into the broader architecture of the parser.

Features

Note_Group Class

- **Grouped Note Storage:** Holds a list of Note objects that are logically grouped.
- **Beamed Flag:** Tracks whether the note group is connected by beams.
- **Tuplet Flag:** Tracks whether the note group is part of a tuplet.
- **String Representation:** Stores a concatenated string of all note representations within the group.
- **Export Utility:** Provides a `to_dict()` method for structured export (e.g., JSON serialization).

Beam Class

- **Beam Status:** Stores the status of the beam (start, continue, or stop), which reflects its role within a beamed group.
- **Beam Number:** Identifies the level or layer of beaming (e.g., first beam, second beam for 32nd notes, etc.).
- **Accessor Methods:** Getters for both beam status and number to enable encapsulated access.

Slur Class

- **Slur Number:** Identifies the slur when multiple are present (important for nested slurs).
- **Slur Type:** Captures the type of slur (start, continue, or stop).
- **Accessor Methods:** Getters for both number and type.

Tie Class

- **Tie Number:** Identifies the tie in multi-tie contexts (though ties are typically simple).
- **Tie Type:** Indicates the tie's function (start, continue, or stop).
- **Accessor Methods:** Getters for both number and type.

Testing

Since these classes are lightweight helper container classes there was no direct testing for them.

Running the Parser

Currently the file path to the mxl file is hard coded in the main function and the output.json file is hardcoded as a parameter in the `save_to_file()` function in the song class to output to the current directory the parser is in. As long as the implemented libraries are installed on the local device the parser will run without issues given a proper file path and a mxl file that conforms to the current supported features and limitations.